

Architecting Game Theory Optimal (GTO) Poker Systems: Bet Sizing, Action Abstraction, and Solver Integration

The transition of poker from a game of human heuristic strategy to one of rigorous mathematical optimization has been driven by the evolution of Game Theory Optimal (GTO) solvers. By defining poker as an extensive-form game of imperfect information, computational models seek to establish a Nash equilibrium—a strategy profile where no player can increase their expected utility by unilaterally deviating. Empirical data demonstrates the overwhelming superiority of this approach; for example, the milestone AI DeepStack achieved an extraordinary win rate of 48.6 Big Blinds (BB) per 100 hands against professional human players in 200BB Heads-Up No-Limit Texas Hold'em (HUNL). This win rate heavily underscores the validity of pursuing a strategy that seeks to minimize expected value (EV) loss to zero.

For the development of a poker training application intended to operate flawlessly across varying degrees of user freedom—specifically divided into a constrained "Simple Mode" and an unconstrained "Standard Mode"—the underlying architecture must accurately reflect equilibrium strategies. This report provides an exhaustive analysis of GTO solver mechanics, the fundamental necessity of range-based thinking, bet sizing methodologies, action abstraction for unlimited bet sizes, action translation algorithms for off-tree decisions, and the real-time subgame solving algorithms required for high-fidelity strategy evaluation.

The Mathematical Foundation: Information Sets and the Necessity of Ranges

Before addressing specific bet sizings or interface constraints, it is critical to answer a fundamental architectural question regarding the solver's perspective: whether the system should utilize ranges. The unequivocal answer is that a GTO system cannot function without range-based evaluations.

In a finite two-player zero-sum extensive-form game with imperfect information, the game tree consists of a set of histories. Because players do not possess perfect information (e.g., they cannot see their opponent's hole cards), these histories are partitioned into information sets. An information set contains all histories that are indistinguishable to the acting player given their observed public and private information. The solver's goal is to compute a behavioral strategy, which defines the probability distribution over the available actions for a player at a specific information set.

Because players do not know the exact cards their opponents hold, they must make decisions against a distribution of all mathematically possible hands the opponent could hold given the preceding sequence of actions. This distribution is known as a range. When

the application grades a user's decision, it must do so not by analyzing the hand in a vacuum, but by evaluating how the user's specific hand functions within their entire range, juxtaposed against the opponent's entire range. Industry-standard data models for range representation include the 13x13 grid matrix, ProPokerTools (PPT) syntax for enumerated combos with percentage weights, and specific preflop macro notations. The solver calculates the expected value of an action based on the combinatorial intersection of these ranges, adjusting for card removal (blocker) effects. Therefore, the engine must track and update the ranges of all active players at every node in the game tree.

GTO Bet Sizing in "Simple Mode"

In "Simple Mode," the application reduces the complex decision tree to three static inputs: Fold, Check/Call, and a single Raise button. Because the user cannot manually input a bet size, the application must dynamically calculate the mathematically optimal raise sizing for the given information set. This requires mapping continuous game states to highly abstracted, predetermined actions based on precomputed GTO heuristics. To maintain accurate alignment with GTO solver outputs, the dynamic calculation for this single button must rely on the concepts of range advantage, nut advantage, and geometric bet sizing.

Preflop Action Space Constraints and Multipliers

Preflop ranges are dictated by positional advantage, stack-to-pot ratio (SPR), and card removal effects. A solver does not infinitely mix preflop bet sizes; rather, it establishes distinct geometric sizing structures that maximize expected value while maintaining a balanced ratio of value hands to bluffs. For the dynamically calculated Raise button in Simple Mode, the preflop sizings should be programmed to standardized GTO defaults based on effective stack depths (typically 100 BB).

The initial open raise, or Raise First In (RFI), should dynamically shift based on the user's position at the table. In early and middle positions, the standard solver sizing is 2.0x to 2.5x the Big Blind. Smaller sizes in early position preserve the stack-to-pot ratio and allow the player to open a wider range of hands while mitigating the mathematical risk of facing 3-bets from players acting behind them with a positional advantage. Conversely, in late positions such as the Button or Small Blind, the primary objective shifts toward stealing the dead money in the blinds. Here, the RFI sizing should be increased to 2.5x to 3.0x the Big Blind, maximizing fold equity against the relatively wide defending range of the Big Blind.

When facing an initial raise, the 3-bet sizing must account for positional dynamics. If the 3-bettor is In Position (IP), the standard GTO multiplier is 3.0x the previous bet. The in-position player can utilize a slightly smaller multiplier because their positional advantage allows them to realize their hand's equity highly efficiently on postflop betting rounds. If the

3-bettor is Out of Position (OOP), the sizing must increase to 4.0x the previous bet. The OOP player must compensate for their informational disadvantage postflop by lowering the SPR and reducing the positional player's maneuverability.

For 4-bets, the geometric scaling changes because the ranges involved become extraordinarily narrow, and the SPR drops rapidly. An IP 4-bet should be sized between 2.2x and 2.5x the previous 3-bet, while an OOP 4-bet requires a slightly larger size of 2.7x to 3.0x to prevent the IP player from flat-calling with speculative hands that possess high implied odds.

Preflop Action	Positional Dynamic	GTO Recommended Sizing	Strategic Justification
Raise First In (RFI)	Early / Middle Position	2.0x – 2.5x Big Blind	Preserves SPR; minimizes risk against IP 3-bets.
Raise First In (RFI)	Late Position (BTN/SB)	2.5x – 3.0x Big Blind	Maximizes fold equity to capture dead money in the blinds.
3-Bet	In Position (IP)	3.0x Previous Bet	Leverages positional advantage for postflop equity realization.
3-Bet	Out of Position (OOP)	4.0x Previous Bet	Neutralizes positional disadvantage by artificially lowering SPR.
4-Bet	In Position (IP)	2.2x – 2.5x Previous Bet	Achieves necessary fold equity with minimal risk given narrow ranges.

Preflop Action	Positional Dynamic	GTO Recommended Sizing	Strategic Justification
4-Bet	Out of Position (OOP)	2.7x – 3.0x Previous Bet	Deters IP calls from speculative, high implied-odds holdings.

Postflop Action Space Constraints and Pot Percentages

Postflop betting strategy diverges fundamentally from preflop heuristics. Bet sizes are no longer strict multipliers of previous bets, but are instead defined as percentages of the total pot. Solvers typically output strategies utilizing discrete fractional representations, such as 33%, 50%, 67%, 100%, and 150%. To map a single Raise button postflop in Simple Mode, the application must analyze the state matrix—identifying the preflop aggressor, positional advantage, and board texture—to default to the highest-frequency GTO bet size for that specific node.

Small bet sizings, typically 25% to 33% of the pot, are the mathematically optimal choice on dry, disconnected flops (e.g., $K\spadesuit 7\heartsuit 2\clubsuit$) or paired boards (e.g., $J\heartsuit J\spadesuit 4\clubsuit$). On these textures, the preflop aggressor retains a massive range advantage, meaning their entire distribution of hands possesses higher average equity than the caller's range. Because the aggressor is incentivized to bet at an extraordinarily high frequency (often 70% to 100% of their range), they must use a small bet size. This mathematically ensures that the high volume of bluffing combinations in their range remains profitable, as a smaller bet requires less fold equity to show a positive expectation.

Large bet sizings, typically 67% to 75% of the pot, are strictly reserved for wet, dynamic board textures (e.g., $9\spadesuit 8\spadesuit 7\spadesuit$) or for turn and river betting rounds. Large bets are symptomatic of a highly polarized strategy. When a solver utilizes a 67% or 75% size, the betting range is composed exclusively of premium value hands (demonstrating a nut advantage) and high-equity semi-bluffs or zero-equity pure bluffs. Mid-strength hands are entirely removed from this betting range and relegated to a checking line. Overbets, ranging from 125% to over 150% of the pot, occur in scenarios where one player possesses the absolute nuts in their range, while the opponent's range is mathematically capped and unable to hold the strongest possible combinations.

When operating in Simple Mode with only one Raise button available, the application must query the solver's strategy distribution for the current information set. GTO strategies are

frequently mixed; for example, the solver might recommend betting 33% of the pot at a 40% frequency and betting 75% of the pot at a 15% frequency, checking the remainder of the time. The application must identify the bet size with the highest probability weight assigned by the solver—in this case, 33%—and dynamically assign that value to the user's interface.

Handling Unlimited Action Spaces in "Standard Mode"

In Standard Mode, allowing the user to bet any amount—or providing a granular slider ranging from 1 BB to an all-in—introduces a critical architectural challenge: the combinatorial explosion of the game tree. The user's query asks whether to keep the unlimited bet sizing, provide multiple choices, or remove it altogether.

The Necessity of Action Abstraction

In No-Limit Texas Hold'em, the action space at any given node is technically bounded only by the player's remaining stack size. If a player has a stack of 10,000 chips, they theoretically have 10,000 distinct bet sizes available. Constructing a game tree with 10,000 branches at every single decision node renders the game computationally intractable for standard solver algorithms.

To solve games of this magnitude, researchers mandate the use of action abstraction. Action abstraction artificially discretizes the infinite action space into a manageable number of allowable actions, forcing the solver to choose only among a predefined set of bet sizes (e.g., 0.5x pot, 0.75x pot, 1.0x pot, and all-in). If the application removes unlimited bet sizing and forces users to select exclusively from these 4 to 5 multiple choices, the GTO evaluation is perfectly accurate and instantaneous because the user is forced to stay "on-tree."

However, removing the unlimited bet slider drastically degrades the realism of the simulation, deviating from how real-world poker is played. The optimal architectural decision for Standard Mode is to retain the unlimited bet sizing on the front end, but implement a complex mathematical translation layer in the backend to bridge the user's custom input with the solver's abstracted game tree.

Action Translation and the Off-Tree Problem

When a user inputs a custom bet size (e.g., 42% of the pot) that does not perfectly align with the solver's abstraction (e.g., the solver only computed branches for 33% and 50%), the application encounters the "off-tree problem". If the application naively rounds the 42% bet to 33%, it permanently distorts the pot odds, stack-to-pot ratio, and geometric progression for all future nodes in that hand. Consequently, subsequent GTO evaluations

for the turn and river become highly inaccurate, as the solver is evaluating a fundamentally different game state.

To evaluate a custom bet efficiently without requiring real-time subgame solving, the engine must utilize an Action Translation Mapping (often referred to as a Reverse Mapping). This algorithmic procedure interprets actions outside the abstraction by mathematically mapping them to adjacent actions within the pre-solved abstraction.

The industry standard methodology for this is the Pseudo-Harmonic Mapping algorithm. Derived from analytical solutions of simplified no-limit poker games, it maps an opponent's arbitrary, off-tree bet size x to one of the nearest abstracted sizes A and B , where $A < x < B$. The probability $f(x)$ that the custom bet x is mapped to the lower abstraction bound A is defined by the following formula:

$$f(x) = \frac{(B - x)(1 + A)}{(B - A)(1 + x)}$$

The corresponding probability of mapping the action to the upper abstraction bound B is simply $1 - f(x)$.

The pseudo-harmonic mapping is theoretically robust because it satisfies Lipschitz continuity. This means that infinitesimally small changes in the user's custom bet size do not result in wildly volatile or discontinuous mappings, ensuring stable feedback. When the user inputs a bet of 42% of the pot, the engine uses this formula to probabilistically map the action to either the 33% node or the 50% node. This permits the evaluation engine to traverse down a pre-solved path to calculate the expected value of the action with remarkably low exploitability.

Nested Subgame Solving for High-Fidelity Custom Bets

While pseudo-harmonic mapping provides an efficient heuristic solution, the most sophisticated and accurate approach to handling off-tree custom bets is Real-Time Subgame Solving, heavily utilized by milestone AIs such as Libratus and Pluribus. Subgame solving, specifically Depth-Limited Solving, guarantees that the system's feedback remains mathematically precise regardless of the user's input.

If a user makes a custom bet that deviates from the tree, the application can freeze the pre-computed blueprint strategy and dynamically generate a brand-new subgame rooted precisely at that custom bet size. First, the engine calculates the exact new pot size and remaining stack sizes resulting from the custom bet, correcting the off-tree distortion. Second, it utilizes the solver's precomputed blueprint strategy to estimate the expected values at the leaf nodes of this new subgame, ensuring that the solver understands the

future implications of the game state without having to simulate all the way to the river. Finally, it performs a rapid real-time nested Counterfactual Regret Minimization solve—often framed as a "resolving" or "max-margin gadget game"—to compute the optimal response to that specific custom bet.

When incorporating a prior strategy into the subgame solve, resolving gadget games often outperform max-margin gadget games because they guarantee safety against exploitation while allowing the solution to combine the strengths of both safe and unsafe solving methodologies. By implementing depth-limited subgame solving, the application can provide perfectly accurate, zero-EV-loss feedback for absolutely any bet size the user chooses, completely bypassing the mathematical compromises inherent in action translation.

Quantifying Play: Expected Value, Odds, and GTO Frequencies

To train a user to become a winning player with zero EV loss, the application must possess a robust scoring engine capable of mathematically defining the quality of a decision. The user asks for a formula to show the odds of each bet performed and how to calculate the odds of an all-in, noting differences between preflop and postflop states.

Expected Value (EV) Calculations and the All-In Scenario

Expected Value is the fundamental metric of poker theory. It represents the average return of an action over an infinite number of iterations. The generic formula for the EV of an action is the sum of the probabilities of each possible outcome multiplied by the payoff of that outcome.

Calculating the odds and EV of an all-in represents a unique mathematical scenario because it eliminates the complexity of future betting rounds. When a player calls an all-in, they no longer have to account for implied odds or reverse implied odds. The EV of calling an all-in is calculated strictly based on current pot odds and raw hand equity. Let P_{pot} be the current size of the dead money in the pot, C be the cost to call the all-in, and E be the player's equity (the statistical probability of their specific hand winning the showdown against the opponent's range).

$$EV_{call} = (E \times (P_{pot} + C)) - ((1 - E) \times C)$$

If $EV_{call} > 0$, the call is mathematically profitable and represents a positive EV decision.

However, there is a profound difference between calculating the odds of an all-in preflop versus postflop. Preflop EV calculations are heavily probabilistic. A hand like $7\spadesuit 6\spadesuit$ might have 40% raw equity against a premium range if both hands are dealt to the river, but in

deep-stacked play, it will rarely realize all of that equity because it will frequently be forced to fold to postflop aggression. Thus, preflop GTO EV cannot rely on raw all-in equity formulas; it is heavily reliant on massive precomputed CFR simulations that account for equity realization. Postflop river decisions, conversely, deal with deterministic equity. Because there are no more cards to come, a hand's equity is either exactly 0% or exactly 100% against specific combinations in the opponent's range, and the calculation depends entirely on blocker effects and the ratio of bluffs to value bets.

Mixed Strategies and the Indifference Principle

The user queries how a formula can show the odds of each bet performed and how the solver calculates what is best when multiple raise options are available. This touches on the most complex aspect of GTO poker: the mixed strategy and the principle of indifference.

In many scenarios, the solver will be entirely indifferent between two or more actions. For example, the solver might calculate that the EV of betting is 4.1 BB, and the EV of checking is exactly 4.1 BB. In this scenario, the solver will employ a mixed strategy—perhaps betting 60% of the time and checking 40% of the time. It mixes frequencies not to increase its own EV (since both yield 4.1 BB), but to remain perfectly unpredictable, ensuring the opponent cannot mathematically exploit them.

If the solver recommends a 60/40 mix, and the user selects the action with the 40% frequency, their EV Loss is precisely zero. Both actions yield the exact same expected value against a perfect opponent. The application must not penalize the user's score for taking the lower-frequency action, provided the EV loss is zero. To provide accurate feedback on multiple bet choices, the UI should display a breakdown of the solver's frequency distribution, allowing the user to understand how to probabilistically balance their range over a large sample size of hands.

EV Loss and Threshold Grading

A GTO solver calculates the expected value of every possible action at a decision node. The optimal action is the one that yields the absolute highest EV. EV Loss is defined as the mathematical difference between the EV of the optimal GTO action and the EV of the action actually chosen by the user.

$$EV_{Loss} = EV_{Optimal} - EV_{UserAction}$$

To provide actionable feedback to the user, the application should translate EV loss into a categorized grading system. Industry standards categorize mistakes based on the number of Big Blinds lost, isolating the magnitude of the error from the raw outcome of the hand.

Grade Category	EV Loss Threshold	Strategic Description
Correct	< 0.5 BB	The action is either optimal or mathematically indistinguishable from optimal. Frequencies may vary, but EV is captured.
Inaccuracy	0.5 – 2.0 BB	A minor deviation. The action might be technically profitable, but leaves a small amount of expected value uncaptured (e.g., an action taken less than 3.5% of the time by the solver).
Mistake	2.0 – 5.0 BB	A significant strategic error. This typically represents a fundamental misunderstanding of range advantage, bet sizing geometry, or equity realization.
Blunder	> 5.0 BB	A catastrophic error. Examples include folding the winning hand, calling a massive overbet with a bluff-catcher that blocks the opponent's bluffs, or shoving a zero-equity hand without fold equity.

Solid regulars typically achieve overall accuracy scores between 90% and 95%, while scores exceeding 95% are considered excellent by GTO standards. Tracking these thresholds over time allows the software to automatically identify leaks based on position, street, and action type.

Algorithmic Engines: CFR and Deep Learning Extensions

To generate the underlying data that feeds the EV and frequency calculations, the application must utilize a solver engine based on Counterfactual Regret Minimization (CFR) or one of its advanced derivatives. CFR solves imperfect-information games by repeatedly traversing the game tree, playing against itself. At each decision node, it computes the instantaneous regret of not taking a specific action.

Let σ be a strategy profile, I be an information set belonging to player i , and Z_I be the terminal histories passing through I . The counterfactual value of I weights each outcome by the probability that everyone except i played to reach it:

$$v_i(I, \sigma) = \sum_{z \in Z_I} \pi_{-i}^\sigma(z) u_i(z)$$

The instantaneous regret of choosing action a over the current strategy σ is calculated as how much better the player would have done by committing to that action:

$$r_t(I, a) = v_i(I, \sigma_{I \rightarrow a}^t) - v_i(I, \sigma^t)$$

The algorithm accumulates this regret over iterations: $R_T(I, a) = \sum_{t=1}^T r_t(I, a)$. By updating the strategy on subsequent iterations to favor actions with high positive regret, the time-averaged strategy mathematically converges to a Nash equilibrium.

Modern CFR Variants for Production

For a production-grade application, standard tabular CFR is highly inefficient. The engine must implement modern algorithmic variants to achieve rapid convergence and manage hardware constraints:

- **CFR+:** This variant floors all negative regrets to zero at each iteration. By preventing the algorithm from accumulating massive negative regrets during early, highly sub-optimal explorations, it allows the strategy to recover and converge significantly faster. It utilizes linear time-averaging weights ($w_t = t$).
- **Discounted CFR (DCFR):** DCFR implements a discounting scheme that down-weights the noisy early iterations and the regrets they produce. It multiplies positive regrets by a discount factor and uses a quadratic average-strategy weight ($w_t = t^2$), allowing the strategy to adapt exceptionally fast in massive game trees.
- **Monte Carlo CFR (MCCFR):** Instead of traversing the entire game tree every iteration, MCCFR samples specific paths, such as External Sampling, to vastly reduce computational overhead while maintaining unbiased estimates of counterfactual values. Recent advancements like CCS-MCCFR (Persistent randomized low-discrepancy chance sampler) locally balance chance outcomes, reducing final exploitability by 19% to 34% across tabular poker formats.
- **Deep CFR and Single Deep CFR:** For deep-stacked games with state spaces too large for manual action abstractions, Deep CFR uses deep neural networks as function approximators. It learns the regret values and strategy profiles without requiring manual state abstraction, relying instead on the network to intrinsically generalize similar board textures and hand strengths. Single Deep CFR simplifies this architecture by removing the need for an average-strategy network, thereby significantly reducing overall approximation error and outperforming standard Deep

CFR in head-to-head exploitability metrics. HDCFR (Hierarchical Deep CFR) further expands this by learning a high-level strategy for skill selection alongside low-level execution components.

Real-Time Application Architecture and Optimization

Delivering a multi-gigabyte GTO dataset to a user interface in milliseconds, while simultaneously performing real-time subgame solving and EV loss calculations, requires a highly optimized technological stack.

Hand Evaluation Performance

At the core of the CFR tree traversals are millions of terminal node evaluations (showdowns) determining which hand wins. A standard 7-card evaluation using brute-force combinatorics is computationally prohibitive. The application should integrate state-of-the-art hand evaluators like the Perfect Hash (PH) Evaluator or Cactus Kev's algorithm.

Rather than iterating through 7-choose-5 combinations, the PH Evaluator relies on a perfect hash algorithm to map any 5, 6, or 7-card hand directly to a pre-computed hash table. This assigns a definitive hand strength integer from 1 (a Royal Flush) to 7462 (the worst possible unsuited high-card hand). Modern implementations built in C++ or Rust (e.g., phevaluator, aya_poker, poker_eval) can achieve performance metrics exceeding 115 million 7-card evaluations per second with a memory footprint of roughly 6 KB to 100 KB. This extreme optimization is absolutely vital for real-time depth-limited subgame solving, where the engine must perform Monte Carlo rollouts instantly.

Suit Isomorphism and Data Interoperability

To reduce the size of the pre-solved database and minimize memory allocation, the application must implement lossless information abstraction via suit isomorphism. In poker, strategic mechanics dictate that a flop of $A\spadesuit K\spadesuit 2\heartsuit$ is mathematically identical to a flop of $A\clubsuit K\clubsuit 2\diamondsuit$. By establishing these equivalence classes, the 22,100 possible unique flops in a deck of cards can be aggressively reduced to exactly 1,755 canonical flops. The application's backend must translate any dealt flop into its corresponding canonical index, query the CFR database, and seamlessly map the output back to the specific suits displayed on the user's screen.

For seamless integration between the solver engine, the database, and the client UI, the application should adopt standardized data schemas. Utilizing the PHH (Poker Hand History) TOML-based format or the Open Hand History (OHH) JSON specification ensures that training sessions, user actions, and solver evaluations are easily parsed and stored.

Standardizing this data model is critical for the long-term persistence tracking required to monitor EV loss, grade blunders, and generate historical performance dashboards.

UI Rendering Architecture

On the front end, rendering a fluid, high-performance poker table with custom chip animations and state transitions should bypass the standard HTML DOM entirely. Standard DOM manipulation relies heavily on the CPU, which becomes constrained when simultaneously processing heavy JavaScript EV calculations. Leveraging a WebGL or WebGPU architecture via engines like PixiJS allows the application to offload all visual rendering to the Graphics Processing Unit (GPU).

A PixiJS state machine can manage the Sprite assets (cards, dealer buttons, particle effects) independently from the React or Vue user interface. This architectural separation ensures steady 60 frames-per-second visual performance and immediate touch-event responsiveness, even while the backend thread is unpacking JSON payloads, executing pseudo-harmonic mappings, and computing instantaneous regrets for custom bet sizings.

Conclusion

Building a GTO poker application that successfully guides users toward a zero EV loss baseline requires an architecture defined by strict mathematical rigor, highly optimized state abstractions, and advanced algorithmic processing.

For Simple Mode, the system must parse underlying GTO distributions to isolate the highest-frequency sizing for a single Raise button. Preflop structures must be anchored to strict positional geometry, utilizing multipliers between 2.0x and 4.0x depending on stack-to-pot ratios and positional advantage. Postflop sizings must dynamically toggle between small range bets (33%) and large polarized bets (67% to 150%) based entirely on board texture and range interactions.

For Standard Mode, the inclusion of unlimited bet sizing is necessary for realism but must be supported by a sophisticated algorithmic bridge. When a user inputs an off-tree custom bet size, the system must either employ the Pseudo-Harmonic Mapping algorithm to translate the bet seamlessly into the solver's predefined action abstraction without losing Lipschitz continuity, or it must trigger a Depth-Limited Subgame Solve. The latter generates a new game tree rooted precisely at the custom bet, computing a new counterfactual regret equilibrium in real time.

By unifying Discounted CFR mechanics, range-based probability tracking, 115-million-per-second hash evaluations, strict EV loss categorization thresholds, and highly optimized PixiJS GPU rendering, the resulting platform will function as an institutional-grade training

environment. This ensures that every decision point is backed by mathematically undeniable expected value computations, fostering true equilibrium-oriented skill development.